C File Synchronization Project

Function-by-function guide for the sender and receiver

This guide explains the roles of the functions in the sender (send_b.c) and receiver implementation shared in the conversation. It focuses on what each function does, how the functions interact, and the protocol fields they exchange.

Scope note: The sender source and networking/protocol helpers were provided as files. The receiver explanation is based on the receiver code pasted in the conversation, including the subsequent deletion and logging changes discussed there. Your local files may differ if you have edited them since sharing them.

SENDER send_b.c	**TCP connection** net.c helpers	**RECEIVER** receive source
Choose upload/delete; serialize request; send bytes; read reply.	send_all() and recv_all() transfer exact byte counts.	Decode header; receive file or delete; log action; reply ACK/ERROR.

1. Sender — send_b.c

The sender runs on the machine that has the files to synchronize. It opens a TCP connection to the receiver, sends one operation per connection, and waits for a response.

send_content(int fd, FILE *f, uint64_t size)

Sends exactly **size** bytes from an already-open file over socket fd.

What it does:

• Uses a 4096-byte buffer to read manageable chunks.

• Tracks remaining bytes so the total sent matches the size announced in the header.

• Calls send_all() for each chunk so partial socket writes are handled.

• Returns -1 if the file read fails/ends early or the socket send fails.

send_file(const char *ip, int port, const char *path)

Uploads one regular file to the receiver.

What it does:

• Uses stat() to check that the path exists and is a regular file, and obtains its size.

• Extracts only the basename (the part after the last slash), so directory paths are not transmitted.

• Opens the file, creates an IPv4 TCP socket, parses the server IP with inet_pton(), and connects.

• Builds an OP_SEND_FILE header containing the operation, filename length, and file size.

• Sends the header, filename bytes, and file contents in that order.

• Reads an 11-byte response header; prints STORED for ACK value 0 or UNCHANGED for nonzero ACK values.

• Closes the socket and file on its success/error paths.

send_delete(const char *ip, int port, const char *path)

Sends a deletion request for a filename. This function was added during the conversation.

What it does:

• Extracts the basename and rejects empty, overlong, '.' and '..' names.

• Connects to the receiver using IPv4 TCP.

• Sends an OP_DELETE header followed by the filename bytes; no file payload is sent.

• Reads the response header and treats ACK value 2 as a successful delete/already-absent response.

• Prints a local error to stderr if connection, transmission, or response handling fails.

main(int argc, char *argv[])

Validates command-line arguments and dispatches the requested operation.

What it does:

• Expects: ./send_b .

• Checks that the port is between 1 and 65535.

• Calls send_file() for upload or send_delete() for delete.

• Returns an exit status of 0 for success and 1 for failure.

Sender command examples

```
./send_b 127.0.0.1 9000 upload test.txt
./send_b 127.0.0.1 9000 delete test.txt
```

The original uploaded send_b.c had only the upload path. The delete function and operation-selecting main() above describe the changes made in the conversation; verify that these are present in your current file.

2. Receiver — receive source

The receiver listens for incoming TCP connections. In the version discussed, it handles one request per connection synchronously, sends a reply, then closes that connection.

compute_checksum(const char *path, uint32_t *out)

Computes a simple 32-bit rolling checksum over a file.

What it does:

• Opens the file in binary mode and reads it in 4096-byte chunks.

• For each byte, updates sum using sum = sum * 31 + byte.

• Writes the result through out and returns 0; returns -1 on open/read error.

Notes: This is not a cryptographic hash, so collisions are possible.

files_identical(const char *a, const char *b)

Checks whether two files appear to have identical contents.

What it does:

• Compares file sizes first using stat().

• If sizes match, computes both rolling checksums and compares them.

• Returns 1 if size and checksum match, otherwise 0.

Notes: A checksum match does not mathematically guarantee identical content.

send_u32(int fd, uint32_t value)

Converts a 32-bit integer to network byte order with htonl() and sends it using send_all().

What it does:

• This is a helper for sending a 32-bit value; the shown receiver flow does not depend on it directly.

recv_u32(int fd, uint32_t *value)

Receives exactly four bytes using recv_all(), converts them from network byte order with ntohl(), and stores the result.

What it does:

• Returns 0 on success and -1 if the receive fails.

safe_name(char *dst, const char *src)

Extracts the basename from a supplied path and validates the resulting filename.

What it does:

• Uses the last slash to strip directory components.

• Rejects empty names, '.' and '..', and names that do not fit MAX_NAME_LEN.

• Copies the accepted basename into dst.

Notes: This flattens paths: two directories containing the same basename map to the same receiver filename.

recv_content(int fd, const char *name, uint64_t size)

Receives the declared file payload and stores it as a file.

What it does:

• Rejects files larger than MAX_FILE_SIZE (100 MiB in the shown code).

• Writes incoming bytes to name.part, receiving exactly size bytes in chunks.

• Deletes the temporary file if receiving, writing, or closing fails.

• If the existing destination file has the same size and checksum, removes the temporary file and returns RES_UNCHANGED.

• Otherwise renames the completed temporary file to the final name and returns RES_STORED.

Notes: A fixed .part name can collide if concurrent uploads target the same name.

handle_delete(int fd, const struct header *h, const char *client_ip)

Reads and validates the filename for an OP_DELETE request, removes the file, and logs the outcome.

What it does:

• Checks name_len, receives exactly that many filename bytes, and adds a terminating NUL.

• Passes the name through safe_name().

• Returns RES_DELETED when remove() succeeds.

• In the updated version, maps errno == ENOENT to RES_NOT_FOUND (already absent); other failures return RES_ERROR.

• Calls log_action() with the client IP, DELETE action, filename, and result when logging is wired in.

handle_file(int fd, const struct header *h, const char *client_ip)

Reads and validates the filename for OP_SEND_FILE, then delegates payload reception.

What it does:

• Checks name_len and receives the raw filename bytes.

• Adds a NUL terminator and validates the name using safe_name().

• Calls recv_content() using the file_size from the header.

• Prints stored/unchanged status and, in the updated version, logs UPLOAD with the IP, filename, result, and size.

• Returns the receive result to handle_client().

handle_client(int fd, const char *client_ip)

Handles one request on an accepted client socket.

What it does:

• Receives exactly HEADER_SIZE bytes (11 bytes in protocol.h).

• Decodes and validates the header with decode_header().

• Dispatches OP_SEND_FILE to handle_file() and OP_DELETE to handle_delete().

• Returns RES_ERROR for invalid headers or unsupported operations.

send_reply(int fd, uint8_t op, uint64_t code)

Builds and sends an 11-byte response header.

What it does:

• Sets the response operation (usually OP_ACK or OP_ERROR), zero name length, and places code in file_size.

• Encodes the header and sends all bytes using send_all().

main(int argc, char *argv[])

Starts the TCP server and processes client connections.

What it does:

• Reads the listening port from argv[1] and validates its range.

• Creates an IPv4 TCP socket, enables SO_REUSEADDR, binds to INADDR_ANY, and listens.

• Accepts a connection and captures the client's IPv4 address using accept() and inet_ntop().

• Passes the socket and client IP to handle_client().

• Sends ACK for stored, unchanged, and deletion outcomes; sends OP_ERROR for other failures.

• Closes the client socket and repeats forever.

log_action(const char *ip, const char *action, const char *name, const char *result, uint64_t size)

Appends an audit record to server.log (the helper added during the conversation).

What it does:

• Opens server.log in append mode.

• Formats the current local timestamp.

• Writes timestamp, client IP, action, filename, result, and byte size.

• Closes the log file after writing.

Notes: The log is written relative to the receiver's current working directory. Sanitize untrusted filenames before logging them.

3. Shared networking and protocol helpers

send_all(int fd, const void *buf, size_t n)

Loops on send() until all n bytes have been sent or an error occurs.

What it does:

• Handles EINTR by retrying.

• Uses MSG_NOSIGNAL to avoid a SIGPIPE signal when sending to a closed peer.

• Returns 0 on full success and -1 on failure.

recv_all(int fd, void *buf, size_t n)

Loops on recv() until exactly n bytes have arrived or the connection fails.

What it does:

• Handles EINTR by retrying.

• Treats recv() returning zero as a closed connection before the requested byte count is complete.

• Returns 0 on full success and -1 on failure.

encode_header(const struct header *h, uint8_t H[])

Serializes the header into exactly 11 bytes: 1-byte op, 2-byte name_len, and 8-byte file_size.

What it does:

• Uses htons() for the 16-bit name length and htobe64() for the 64-bit size/value.

• Copies each field into the byte buffer explicitly instead of sending a C struct directly.

decode_header(const uint8_t *buf, struct header *h)

Parses the 11-byte wire header into a struct header.

What it does:

• Copies the integer fields out of the byte buffer and converts them to host byte order.

• Checks that the operation code is in the allowed range, the filename length is within MAX_NAME_LEN, and file operations have nonzero names.

• Returns -1 for invalid headers.

Protocol fields

Field	Size	Meaning
op	1 byte	Operation: OP_SEND_FILE, OP_DELETE, OP_ACK, OP_ERROR
name_len	2 bytes	Filename byte count
file_size	8 bytes	File payload size in upload requests; result code in ACK responses

4. End-to-end request flow

Upload: Sender connects → sends header + filename + file bytes → receiver writes .part → checks whether unchanged → stores by rename → logs → ACK.

Delete: Sender connects → sends OP_DELETE header + filename → receiver validates name → removes file or recognizes it is absent → logs → ACK.

Transport: TCP is a byte stream. send_all() and recv_all() ensure each fixed-size header/name/payload segment is transferred completely, unless the connection fails.

5. Important limitations to remember

• The receiver handles one request at a time; a slow client blocks later clients.

• The current basename-only naming scheme does not preserve directory trees and can cause same-name collisions.

• The rolling checksum can collide; it is suitable only as a simple comparison shortcut, not a strong integrity guarantee.

• A successful TCP transfer does not provide authentication or encryption. Do not expose this server to untrusted networks without adding protections.

• The exact function signatures and log calls should be checked against the latest local source, because the receiver was edited interactively after the uploaded snapshot.